

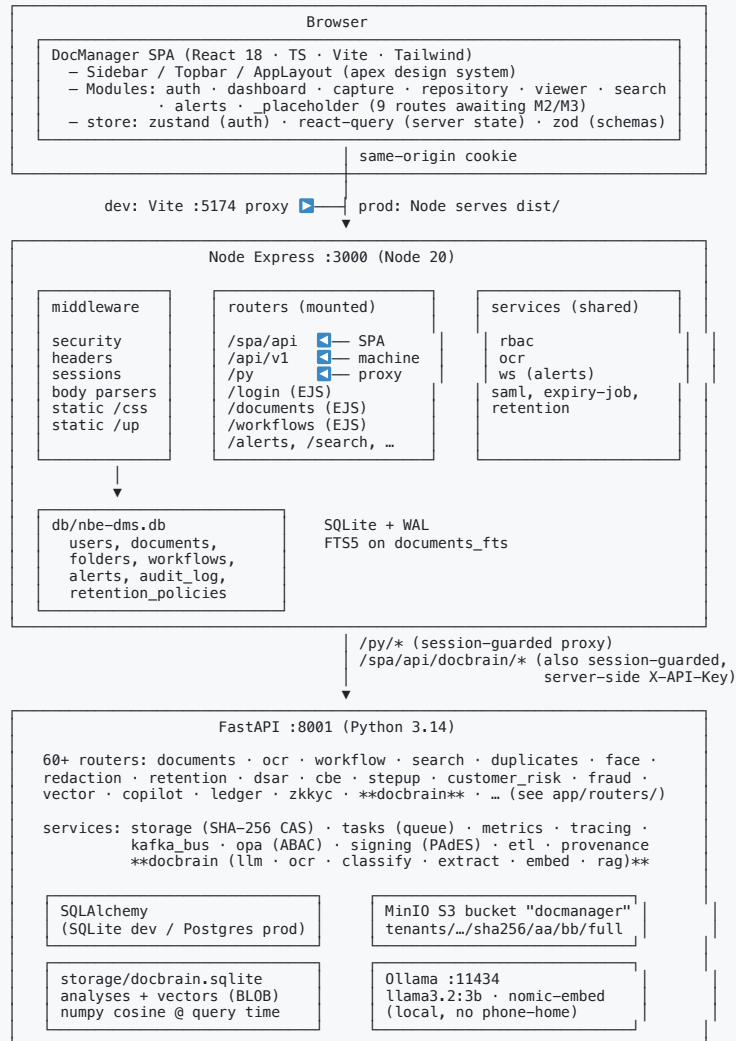
DocManager — Architecture

How the system is organised, how requests flow, and where the boundaries are.

Companion docs: [TECHNICAL.md](#) · [PROJECT.md](#).

Last updated: 2026-04-17

1. Component diagram

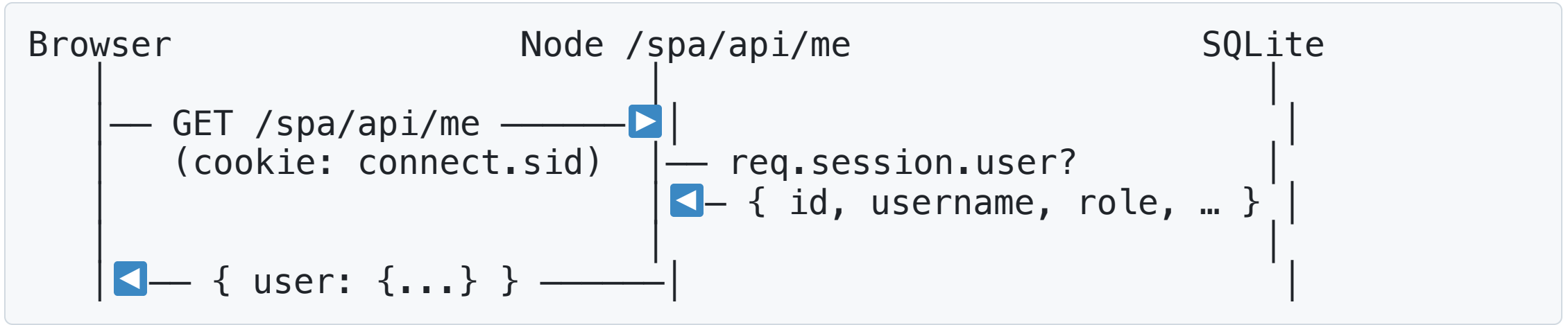


2. Layers, by responsibility

Layer	Owner of	Does NOT own
SPA	Presentation, client routing, form UX, RBAC nav hiding, local form validation	Authoritative RBAC, persistence, business rules
Node spa-api	Session auth, SPA JSON contract, SQL queries against Node SQLite, file uploads, FTS5 search	OCR, duplicate detection, workflow state machine, ABAC
Node /py proxy	Session gate + request forwarding	Any business logic
Node legacy EJS	Server-rendered mirror of the old UI (still live, no longer the default)	New features
	Backend business logic, OCR identification	

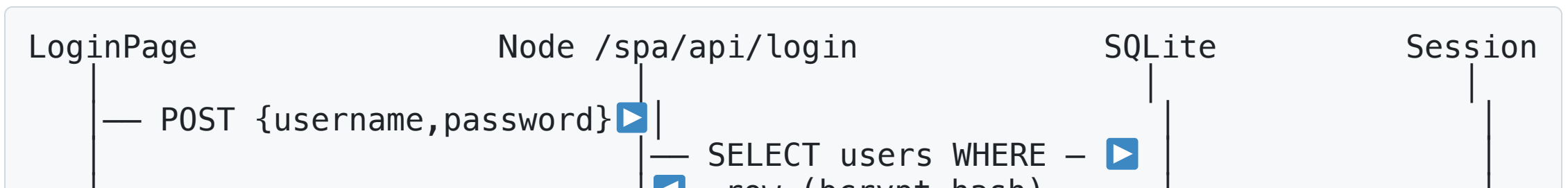
3. Data flow

3.1 Auth hydration (SPA boot)



Store transitions: `status: 'unknown' → 'authenticated' | 'guest'.`

3.2 Login



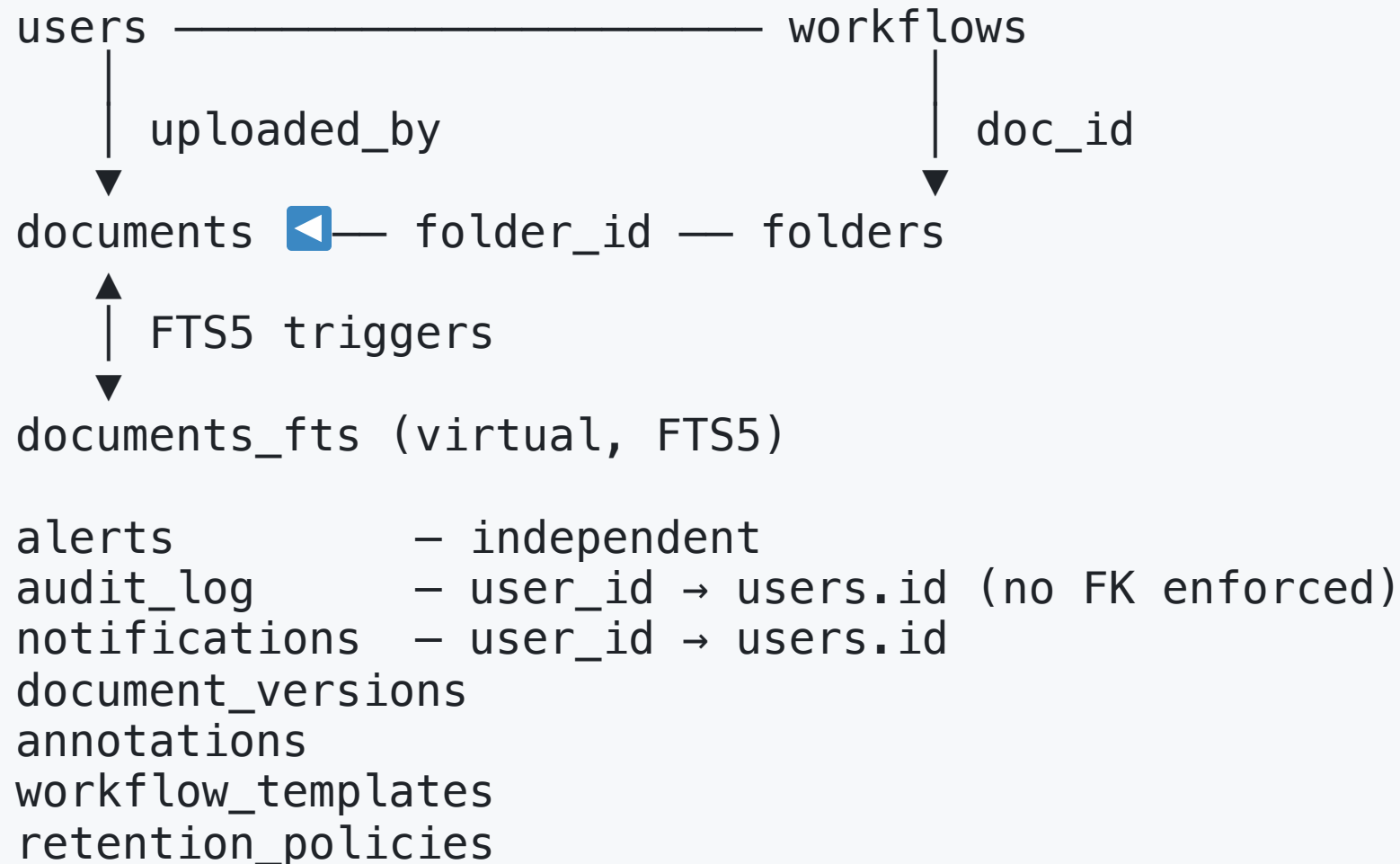
4. Routing

4.1 SPA routes

<code>/login</code>	→ LoginPage (split-screen carousel)
<code>/</code>	→ DashboardPage
<code>/capture</code>	→ CapturePage
<code>/repository</code>	→ RepositoryPage
<code>/viewer</code>	→ ViewerPage (no id – instructional)
<code>/viewer/:id</code>	→ ViewerPage (loads document)
<code>/search</code>	→ SearchPage
<code>/alerts</code>	→ AlertsPage
<code>/indexing, /workflows, /ai, /reports, /compliance, /integration, /security, /users, /admin</code>	→ ComingSoonPage (M2 / M3)
<code>*</code>	→ redirect /

5. Persistence

Node SQLite (`db/nbe-dms.db`)



6. RBAC & branch scoping

Three independent mirrors — all must stay in sync:

Layer	File	Roles	Permissions
Node	services/rbac.js	Doc Admin , Maker , Checker , Viewer	capture , index , approve , reject , admin , security , delete , upload , view , workflow
			capture , index ,

7. Styling system

Single source of truth: [tailwind.config.ts](#) — tokens ported from `apex_core_cbs/UI_AGREEMENT.md §2`. See [TECHNICAL.md §5](#) for the full table.

Enforcement:

- No raw hex in `.tsx` (except in `tokens.ts` itself and in the login carousel's inline styles for dynamic CSS variables).
- Component classes (`.btn-primary`, `.card`, `.module-label`, ...) centralised in `styles/index.css`.
- Icons: `lucide-react` only, never a different icon set.
- Animations for the login carousel live in `styles/index.css` under `.auth-*` classes (perspective rotate + blur + drift + grid pan).

8. Deployment targets

Env	Where
Local dev	<code>./start.sh</code> — Python :8001 + Node :3000 + Vite :5174
Container (current)	Dockerfile runs Node + SQLite in a single image
Python container	python-service/Dockerfile + <code>docker-compose.yml</code>
K8s (planned)	python-service/helm/nbe-dms + python-service/terraform/

For the SPA: `vite build` produces `apps/web/dist/` (~224 KB gzipped). Production serving plan: add `app.use(express.static('apps/web/dist'))` to [server.js](#) and a catch-all to serve `index.html` for SPA routes. Not yet wired — the dev server is the current delivery path.

9. Operational concerns

- **Logs:** `.run/{node,python,web}.log` (gitignored).
- **Health checks:** Node `/login` (200), Python `/health` (200), SPA `/` (200).
- **Background jobs:** `services/expiry-job.js` (daily 02:00), `services/retention.js` (daily 02:30). Both start inside `server.js`.
- **Observability (Python):** Prometheus middleware on `/metrics`, OpenTelemetry tracing initialisable via `services/tracing.py`.
- **Restart script:** `./restart.sh` stops all three, then starts them.

10. DocBrain layer (what ships in the pilot)

The target AI architecture is in [AI_STRATEGY.md](#) — vLLM, Qdrant, BGE-M3, LangSmith. That is the destination. This section is what **actually runs** today, and the intended migration path:

Concern	Pilot (today)	Target
Chat model	Ollama + llama3.2:3b	vLLM + Llama 3.1 8B/70B + Qwen-2 (Arabic)
Embeddings	Ollama + nomic-embed-text (768-dim)	ONNX BGE-M3 (1024-dim, multilingual)
Vector store	docbrain_vectors (SQLite BLOB) + numpy cosine	pgvector (pooled) / Qdrant (silo+)
		...

11. Extension points

Adding a new module (e.g. "Indexing"):

1. Create `src/modules/indexing/api.ts` with zod schemas + axios calls.
2. Create `src/modules/indexing/IndexingPage.tsx`.
3. Replace the placeholder in [App.tsx](#): `<Route path="/indexing" element={ <IndexingPage /> } />`.
4. Add server endpoints to [routes/spa-api.js](#) (session-authenticated, RBAC-checked).
5. Add a Playwright spec under `apps/web/e2e/`.
6. Tick the `comingSoon: true` flag off in `components/layout/nav.ts`.

No cross-module imports. If you need shared behaviour, lift it into `lib/`, `components/ui/`, or `components/layout/`.